

- OS
- ↳ In order to virtualize the CPU, OS needs to somehow share the physical CPU among many jobs. This is called time-sharing.
 - ↳ All ready processes are in a queue
 - ↳ The OS will choose process A to run on the CPU
 - ↳ After sometime, it'll take back the control of CPU from process A
 - ↳ Schedule the next process (B) to run on the CPU & put process A back in queue & so on.
 - ↳ CPU run much faster than humans realize so this creates an illusion that they are running at the same time.

Two concerns:

1. How can we implement virtualization without adding excessive overhead? (Performance)
2. How can we run processes efficiently while maintaining control over the CPU? (Control)

↳ OS needs to have control as it is in charge of resources.

Without it, a process could run forever, take over the machine & access information that it should not be allowed to access.

One way we could solve,

A program requests OS for output of its instruction. This way OS maintains complete control but performance is affected.

Limited Direct Execution

↳ Run the program directly on the CPU.

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute call main()	Run main()
	Execute return from main
Free memory of process	
Remove from process list	

Figure 6.1: Direct Execution Protocol (Without Limits)

Challenges:

- How do we make sure program doesn't do anything that we don't want to do, while still running it effectively
- When we are running the process, how does the OS stop it running & switch to another process.

Problem #1: Restricted Operations

THE CRUX: HOW TO PERFORM RESTRICTED OPERATIONS

A process must be able to perform I/O and some other restricted operations, but without giving the process complete control over the system. How can the OS and hardware work together to do so?

Why do system calls look like procedure calls?

↳ OS code

↳ function call in user code

It is a procedure call, but hidden inside it is the trap instruction.

When you call `open()` in C, you are calling a procedure call into the C library.

Then the library uses agreed-upon calling convention with the kernel to put the arguments into well known locations along with the system call's

Id like a stack or registers.

Essentially, we are passing the arguments from C's `open()` to OS' `open()`.

Then, execute the trap instruction. The code in the library unpacks return values & returns control to the user program.

↳ A naive approach would be to let the user code do whatever it wants but we know that's not good.

↳ So, we introduce a new processor mode known as user mode; code that runs in this mode is restricted in what it can do.

For example,

Code in this mode can't issue I/O requests

↳ In contrast is kernel mode, which the OS (or kernel) runs in.

In this mode, there are no restrictions.

So, how can a user process issue such requests?

↳ To enable this, we have system calls that only expose certain key piece of functionality to user program i.e. some system calls.

↳ To execute a system call,

- A program must execute a special trap instruction
- This instruction raises execution mode (user mode $\rightarrow$ kernel mode)
- The system can perform whatever privileged operations are needed & thus do the work for the calling process.

- When finished, the OS calls a special return-from-trap instruction to return back into the calling process.

↳ The hardware needs to save enough of caller's registers to be able to return correctly.

For example,

On x86, the processor will push the PC, flags & few other registers onto a per-process kernel stack.

The return-from-trap will pop off these values & resume execution of the user mode program.

So, how does the trap know which code to run inside the OS?

↳ Clearly, the calling process can't specify an address to jump to (as you would in a procedure call), doing so would allow programs to jump anywhere into the kernel.

↳ The kernel must carefully control what code executes upon a trap.

The kernel does this by setting up a trap table at boot time.

↳ A machine boots up in kernel mode & can configure machine hardware.

↳ One of the first thing OS does is tell the hardware what code should run when certain exceptional events occur.

↳ In a nutshell, the OS informs the hardware about the locations of these trap handlers.

TIP: BE WARY OF USER INPUTS IN SECURE SYSTEMS

Even though we have taken great pains to protect the OS during system calls (by adding a hardware trapping mechanism, and ensuring all calls to the OS are routed through it), there are still many other aspects to implementing a **secure** operating system that we must consider. One of these is the handling of arguments at the system call boundary; the OS must check what the user passes in and ensure that arguments are properly specified, or otherwise reject the call.

For example, with a `write()` system call, the user specifies an address of a buffer as a source of the write call. If the user (either accidentally or maliciously) passes in a "bad" address (e.g., one inside the kernel's portion of the address space), the OS must detect this and reject the call. Otherwise, it would be possible for a user to read all of kernel memory; given that kernel (virtual) memory also usually includes all of the physical memory of the system, this small slip would enable a program to read the memory of any other process in the system.

In general, a secure system must treat user inputs with great suspicion. Not doing so will undoubtedly lead to easily hacked software, a despairing sense that the world is an unsafe and scary place, and the loss of job security for the all-too-trusting OS developer.

↳ To specify the exact system call, a system call number is usually assigned to each system call.

The user code is responsible for placing the system call number on a specified location (register or stack)

↳ Figure 6.2 summarizes this protocol.

We assume each process has its own kernel stack

There are two phases in LDE.

1. At boot time, the kernel initializes the trap table, and the CPU remembers its location
2. When running a process, the kernel allocates node on process list, memory etc before using return-from-trap instruction to start the execution process

This switches the CPU to user mode & begins running the process. When the process wishes to issue a system call, it traps back into the OS, which handles it & once again returns control

via return-from-trap to the process.

When the process finishes, it returns from main() by calling exit() system call.

Then, at last, the OS cleans up & we are done

OS @ boot (kernel mode)	Hardware	
initialize trap table	remember address of... syscall handler	
OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list Allocate memory for program Load program into memory Setup user stack with argv Fill kernel stack with reg/PC		
return-from-trap	restore regs (from kernel stack) move to user mode jump to main	↳ In this way, return from trap can be used for both
Handle trap Do work of syscall		Run main() ... Call system call trap into OS
return-from-trap	save regs (to kernel stack) move to kernel mode jump to trap handler	
	restore regs (from kernel stack) move to user mode jump to PC after trap	
Free memory of process Remove from process list		... return from main trap (via exit())

Figure 6.2: Limited Direct Execution Protocol

Problem #2: Switching b/w processes

How can the OS switch processes if OS itself isn't running during user process?

THE CRUX: HOW TO REGAIN CONTROL OF THE CPU

How can the operating system **regain control** of the CPU so that it can switch between processes?

A Cooperative Approach: Wait for system calls

↳ In this style, the OS trusts processes of the system to behave reasonably. Processes that run for too long are assumed to periodically give up the CPU so that OS can decide to run some other task.

↳ Most processes transfer control of CPU to the OS by making system calls or when they do some thing illegal like divide by 0.

A Non-cooperative Approach: The OS takes control

THE CRUX: HOW TO GAIN CONTROL WITHOUT COOPERATION

How can the OS gain control of the CPU even if processes are not being cooperative? What can the OS do to ensure a rogue process does not take over the machine?

↳ The answer is timer interrupt. A timer device can be programmed to an interrupt every so many ms.

↳ When the interrupt is raised, the currently running process is halted, & pre-configured interrupt handler in the OS runs.

↳ At this point, the OS has regained control & can do whatever it wants i.e stop the current process & start a different one.

↳ The OS must inform the hardware of which code to run when timer interrupt occurs & this is done at boot time.

↳ The hardware has some responsibilities when an interrupt occurs:

Saving & Restoring Context

↳ Once the OS has control again, it needs to decide whether to continue running the current process or switch to a different one.

This decision is made by part of OS called scheduler.

↳ If the decision is to switch, the OS executes context-switch.

Essentially, all the OS has to do is to save a few registers values for the current process & restore few for the soon-to-be-executed process.

By doing so, the return-from-trap instruction returns to another process instead.

↳ Figure 6.3 shows a timeline of this,

1. Times interrupt:

- Hardware saves Process A's user-mode registers (AC, PC, flags) onto A's kernel stack.
 ← Save 01 done by hardware
- Switches to kernel mode
- Jump to trap handler

2. Trap handler

- ↳ Jump to Times interrupt handler
- OS decides what to do (context switch to Process B)
- OS calls switch() routine.

3. Switch() routine:

- OS saves the current kernel registers (kernel SP) into Process A's PCB
 ← Save 02

done by OS

- Restores Process B's kernel context from Process B's PCB.
- Change kernel SP to Process B's kernel SP.

4. Return-from-trap:

- Restores Process B's user-mode context (AC, SP, PC, flags) from Process B's kernel stack
- Switch to user mode
- Jump to B's PC

OS @ boot (kernel mode)	Hardware	
initialize trap table	remember addresses of... syscall handler timer handler	
start interrupt timer	start timer interrupt CPU in X ms	
OS @ run (kernel mode)	Hardware	Program (user mode)
		Process A
		...
	timer interrupt save regs(A) → k-stack(A) move to kernel mode jump to trap handler	
Handle the trap Call switch() routine save regs(A) → proc_t(A) restore regs(B) ← proc_t(B) switch to k-stack(B) return-from-trap (into B)		
	restore regs(B) ← k-stack(B) move to user mode jump to B's PC	
		Process B
		...

Figure 6.3: Limited Direct Execution Protocol (Timer Interrupt)

↳ Figure 6.4, The context structures old & new are found in the old & new process's process structure.

```
1 # void switch(struct context *old, struct context *new);
2 #
3 # Save current register context in old
4 # and then load register context from new.
5 .globl switch
6 switch:
7     # Save old registers
8     movl 4(%esp), %eax # put old ptr into eax
9     popl 0(%eax)       # save the old IP
10    movl %esp, 4(%eax)  # and stack
11    movl %ebx, 8(%eax)  # and other registers
12    movl %ecx, 12(%eax)
13    movl %edx, 16(%eax)
14    movl %esi, 20(%eax)
15    movl %edi, 24(%eax)
16    movl %ebp, 28(%eax)
17
18    # Load new registers
19    movl 4(%esp), %eax # put new ptr into eax
20    movl 28(%eax), %ebp # restore other registers
21    movl 24(%eax), %edi
22    movl 20(%eax), %esi
23    movl 16(%eax), %edx
24    movl 12(%eax), %ecx
25    movl 8(%eax), %ebx
26    movl 4(%eax), %esp # stack is switched here
27    pushl 0(%eax)      # return addr put in place
28    ret                # finally return into new ctxt
```

movl src, dest
popl (pop from stack)
% indicates value in the register

Figure 6.4: The xv6 Context Switch Code

```
c
// Alternative C-style pseudocode
void switch(struct context *old, struct context *new) {
    // Save current context to old
    old->ip = current_instruction_pointer;
    old->esp = stack_pointer;
    old->ebx = ebx;
    old->ecx = ecx;
    old->edx = edx;
    old->esi = esi;
    old->edi = edi;
    old->ebp = ebp;

    // Load new context from new
    ebp = new->ebp;
    edi = new->edi;
    esi = new->esi;
    edx = new->edx;
    ecx = new->ecx;
    ebx = new->ebx;
    stack_pointer = new->esp; // Critical: switch stacks here

    // Jump to new context
    jump_to(new->ip);
}
```